





【C++ 筆記】模板 (Templates) (上) - part 31

目錄 (Table of Contents) :

- 先搞懂泛型程式設計
- C++ 模板定義
 - 函數模板
 - 類別模板
- 建立模板實例 (instance)
- 多個資料型態的模板
- 變數模板 (C++ 14).
- 預設模板參數
- 總結
 - 泛型程式設計 (Generic Programming)
 - 函數模板 (Function Templates)
 - 類別模板 (Class Templates)
 - 建立模板實例 (instance)
 - 多個資料型態模板
 - 變數模板 (C++14 起)
 - 預設模板參數
- 參考資料

很感謝你點進來這篇文章。

你好，我並不是什麼 C++、程式語言的專家，所以本文若有些錯誤麻煩請各位鞭大力一點，我極需各位的指正及指導！！本系列文章的性質主要以詼諧的口吻，一派輕鬆的態度自學程式語言，如果你喜歡，麻煩留言說聲文章讚讚吧！

先搞懂泛型程式設計

泛型程式設計 (Generic programming) ，是一種程式設計的風格或範式 (paradigm) 。

註：最大名鼎鼎的 OOP 也是一種程式設計的範式。



泛型允許程式設計師在強型別程式設計語言中編寫代碼時使用一些以後才指定的類型，在實例化時作為參數指明這些類型。各種程式語言和其編譯器、執行環境對泛型的支援均不同。Ada、Delphi、Eiffel、Java、C#、F#、Swift 和 Visual Basic .NET 稱之為泛型 (generics) ；ML、Scala 和 Haskell 稱之為參數多型 (parametric polymorphism) ；C++ 和 D 稱之為模板。具有廣泛影響的1994年版的《Design Patterns》一書稱之為參數化類型 (parameterized type) 。

From Wikipedia

嗯...簡單來說，泛型在 C++ 中稱為模板，而模板的意思可以概括為「一種用來定義函數或類別的語法機制，可讓資料型態在使用時再指定，從而支援多種資料型態的操作」。

如 C++ 的 `vector` 即為模板的最經典例子，而 `vector` 可以有 `vector<int>`，也可以是 `vector<string>`。

另一個例子像是 `sort()` 函數，用模板可以讓它適應多種不同的資料型態，用在 `vector<int>` 的話就是將數字升序排序；用在 `vector<string>` 的話就是依照字典序排序。

用模板的好處就是可以讓 `sort()` 更好維護跟編寫，不用再為每個不同型態的參數再寫一次函數定義。

C++ 模板定義

函數模板

```
1 | template <typename type> ret-type func-name(parameter list)
2 | {
3 |     // the body of function
4 | }
```



typename 是 C++ 關鍵字。(改成 class 也可以)

type 是一個 placeholder，可為任意名稱，這個名稱代表資料型態，可以是 int，也可以是 string 等等。

ret-type 為函數的回傳型態。

func-name 為函數名稱。

(parameter list) 為參數列表。

以下是一個簡單範例：



```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  template <typename T>
6
7  T add(T a, T b) {
8      return a + b;
9  }
10
11  int main(){
12      int a = 100, b = 50;
13
14      float f1 = 10.0, f2 = 2.8;
15
16      string s1 = "123", s2 = "456";
17
18      cout << add(a, b) << endl;
19
20      cout << add(f1, f2) << endl;
21
22      cout << add(s1, s2) << endl;
23
24      return 0;
25  }
```

Output :

```
150
12.8
123456
```

以上範例將兩變數相加製作一個函數模板，可支援多種不同的資料型態，數字相關的資料型態就是做相加，而字串則為串接。

類別模板

```
1  template <class type> class class-name {
2      .
3      .
4      .
5  }
```

<class type> 的 class 也可寫 typename 。

class-name 為類別名稱。

範例：



```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  template <class T> class Box{
6  public:
7      T value;
8
9      Box(T v){
10         value = v;
11     }
12
13     void show(){
14         cout << "Value : " << value << endl;
15     }
16 };
17
18 int main(){
19     Box <int> int_Box(100);
20
21     Box <string> str_Box("LukeTseng");
22
23     Box <float> flo_Box(87.87);
24
25     int_Box.show();
26
27     str_Box.show();
28
29     flo_Box.show();
30
31     return 0;
32 }
```

Output：

```
Value : 100
Value : LukeTseng
Value : 87.87
```

上述範例建立一個 **Box** 類別模板，透過模板可以支援不同的資料型態，並在 `show()` 中輸出對應的資料型態之值。（`int` 就輸出 `100`，`string` 就輸出 `LukeTseng`，`float` 就輸出 `87.87`）

建立模板實例 (instance)

語法：



`name_of_entity<type1, type2, ...>`

- `name_of_entity`：模板名稱。

將函數模板的範例拿來改一改：

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  template <typename T>
6
7  T add(T a, T b) {
8      return a + b;
9  }
10
11  int main(){
12
13      cout << add<int>(100, 50) << endl;
14
15      cout << add<float>(10.0, 2.8) << endl;
16
17      cout << add<string>("123", "456") << endl;
18
19      return 0;
20 }
```

Output：

```
150
12.8
123456
```


多個資料型態的模板

就是多加幾個 `<class T1, class T2 ...>` 。



```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  template <class T1, class T2, class T3> class Box{
6  public:
7      T1 value1;
8      T2 value2;
9      T3 value3;
10
11
12      Box(T1 v1, T2 v2, T3 v3) : value1(v1), value2(v2), value3(v
13
14      void show(){
15          cout << "Value1 : " << value1 << endl;
16          cout << "Value2 : " << value2 << endl;
17          cout << "Value3 : " << value3 << endl;
18      }
19  };
20
21  int main(){
22      Box <int, float, string> BBB(88, 87.87, "LukeTseng");
23      BBB.show();
24      return 0;
25  }
```

Output :

```
Value1 : 88
Value2 : 87.87
Value3 : LukeTseng
```

變數模板 (C++ 14)

C++ 14 開始才有變數模板。

語法：

```
template <typename T> constexpr T pi = T(3.14159);
```

pi 即為變數模板。

constexpr 是 C++ 11 開始才有的關鍵字，為常數運算式，該變數或函數的值能夠在編譯階段 (compile-time) 被求值。



編譯期求值顧名思義就是在程式編譯階段的時候，編譯器就能對某些運算式或函數進行計算，不用等到程式執行時才被計算。

而 const 則不一定。

constexpr 在變數模板上通常會加，也可以不加，只是會比較不安全、效率較低，會有多重定義而已。

範例：

```
1  #include <iostream>
2  #include <iomanip>
3
4  using namespace std;
5
6  // 變數模板, pi 圓周率常數
7  template<typename T> constexpr T PI = T(3.14159265358979323846)
8
9  // 函數模板, 計算圓面積
10 template<typename T> T area(T r) {
11     return PI<T> * r * r;
12 }
13
14 int main() {
15     cout << fixed << setprecision(15);
16     cout << "PI<int>: " << PI<int> << ", area(2): " << area(2)
17     cout << "PI<float>: " << PI<float> << ", area(2.0f): " << a
18     cout << "PI<double>: " << PI<double> << ", area(2.0): " <<
19     return 0;
20 }
```



Output :

```
PI<int>: 3, area(2): 12
PI<float>: 3.141592741012573, area(2.0f): 12.566370964050293
PI<double>: 3.141592653589793, area(2.0): 12.566370614359172
```

`cout << fixed << setprecision(15);` 為控制 `cout` 小數位數的指令，不然原本預設都只能顯示五位。



這部分也可以看出 `float` 與 `double` 的精度差距，`float` 只能精準顯示到 6 位，後面就亂掉了，`double` 可以顯示到 15 或 16 位 (依照 OS、編譯環境決定) 。

預設模板參數

沿用類別模板的範例，預設模板參數僅需指定資料型態即可：



```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  template <typename T1, typename T2 = double, typename T3 = stri
6  public:
7      T1 value1;
8      T2 value2;
9      T3 value3;
10
11
12      Box(T1 v1, T2 v2, T3 v3) : value1(v1), value2(v2), value3(v
13
14      void show(){
15          cout << "Value1 : " << value1 << endl;
16          cout << "Value2 : " << value2 << endl;
17          cout << "Value3 : " << value3 << endl;
18      }
19  };
20
21  int main(){
22      Box <int, float, string> B1(88, 87.87, "LukeTseng");
23      B1.show();
24      cout << "-----" << endl;
25      Box <int> B2(88, 87.87, "LukeTseng");
26      B2.show();
27      return 0;
28  }
```

Output :

```
1  Value1 : 88
2  Value2 : 87.87
3  Value3 : LukeTseng
4  -----
5  Value1 : 88
6  Value2 : 87.87
7  Value3 : LukeTseng
```

可發現 B2 模板在只有宣告一個 `int` 的情況下，也可輸入後面的值，但這後面的值要與預設模板參數的資料型態相同（是 `double` 就寫 `double` 型態的值）。

繼續講下去就又是更多花樣、進階的模板了～礙於篇幅原因先就此作結。

總結



泛型程式設計 (Generic Programming)

是一種程式設計範式，允許型態於編譯期決定，支援多種型態的資料操作。

在 C++ 中稱為「模板 (Templates)」。

函數模板 (Function Templates)

語法：

```
1 | template <typename T>
2 | T func(T a, T b) {
3 |     return a + b;
4 | }
```

- `T` 為型態參數，於函數使用時才具體指定（根據輸入的參數之資料型態去決定）。
- 可用 `typename` 或 `class` 表示型態參數。

類別模板 (Class Templates)

```
1 | template <class T>
2 | class Box {
3 |     T value;
4 | };
```

- `T` 是類別中的型態成員。
- `<class T>` 也可表示成 `<typename T>` 皆可，只要在他後面是 `class` 關鍵字就可以是一個類別模板。

建立模板實例 (instance)

語法 (取用函數模板的 `add(a, b)` 範例做示範) :

```
add<int>(100, 50);
```

多個資料型態模板



語法 :

```
template <class T1, class T2, class T3>
```

- 本篇範例中 `Box<T1, T2, T3>` 可同時處理三種不同的資料型態。

變數模板 (C++14 起)

語法 :

```
template <typename T> constexpr T PI = T(3.14159);
```

預設模板參數

語法 :

```
template <typename T1, typename T2 = double, typename T3 = string, ..
```



- 預設了型態後，若使用時只有指定 `T1`，後面兩個就要對應輸入型態值了。

參考資料

[Generic programming - Wikipedia](https://en.wikipedia.org/wiki/Generic_programming)

(https://en.wikipedia.org/wiki/Generic_programming).

Day 5 <泛型>到底是甚麼鬼? | iT 邦幫忙::一起幫忙解決難題，拯救 IT 人的一天 (<https://ithelp.ithome.com.tw/m/articles/10295042>).

Template (C++) - Wikipedia ([https://en.wikipedia.org/wiki/Template_\(C%2B%2B\)](https://en.wikipedia.org/wiki/Template_(C%2B%2B))).

Templates in C++ - GeeksforGeeks

(<https://www.geeksforgeeks.org/cpp/templates-cpp/>).

C++ 模板 | 菜鸟教程 (<https://www.runoob.com/cplusplus/cpp-templates.html>).

